

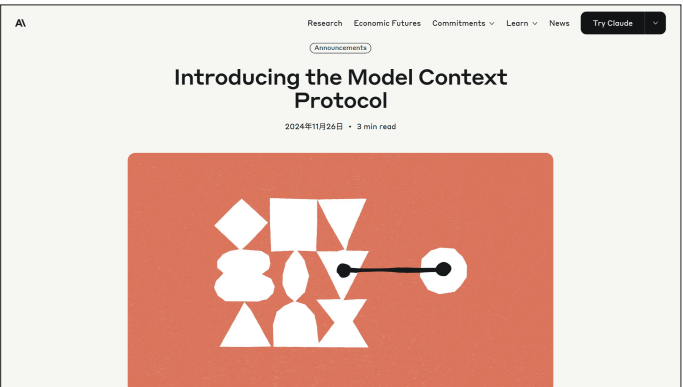
第 1 章

MCPとは

Model Context Protocol

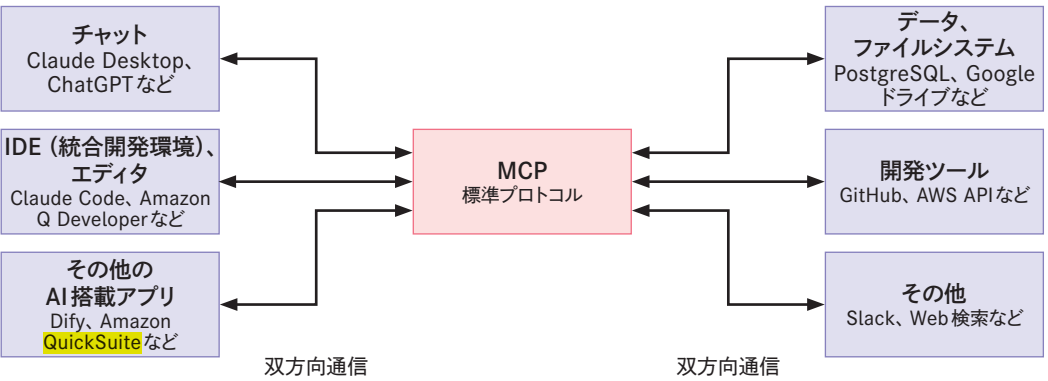
MCPは**Model Context Protocol**の略称で、2024年11月にAnthropic社がオープンソースとして公開した規格（プロトコル）です^{注1.1}。AIアプリを外部システムに接続するための規格で、MCPを利用することで、さまざまなデータソースやツールと連携してタスクを実行できるようになります。

■ Introducing the Model Context Protocol



MCPはしばしば「AIアプリのためのUSB Type-C（以下、USB-C）ポート」と例えられます。USB-Cを利用するとさまざまな機器を標準化されたケーブル一本で接続できるのと同様に、MCPを利用すると、標準化されたプロトコルでAIアプリを外部システムに接続できるようになります。

■ MCPを通じてさまざまなデータソースやツールと連携できる



MCPが公開される以前は、AIアプリを開発する際、アプリごとに個別の連携機能を開発する必要がありました。

注 1.1 <https://www.anthropic.com/news/model-context-protocol>

たとえば、ファイルシステムへのアクセスやデータベース接続、外部API呼び出しなどの同じような機能を何度も実装していました。MCPの登場により、このような実装は不要となり、開発者は**ビジネスロジックの開発に集中**できます。

本書では、MCPが登場した背景と解決する課題、そしてAWSのエコシステム上における活用方法を詳しく解説していきます。

まず第1章では、MCPが登場するまでの技術的背景から見ていきましょう。

1.1 MCP 登場までのバックグラウンド

2022年11月30日にOpenAI社がChatGPTとGPT-3.5をリリースしました^{注1.2}。本書を手にった皆さんには説明は不要かもしれませんが、ChatGPTはチャット形式のインターフェイスを使いLLM (Large Language Model：大規模言語モデル) とやりとりができるアプリです。ChatGPTの登場以降、LLMを含めた生成AIがITやビジネスの領域で大きなトレンドとなっており、2026年でもなお話題の中心にあります。

本節では、ChatGPTの登場から2025年末までのLLMにまつわる主なトピックを振り返り、MCPの登場とその普及までの背景をまとめます。それぞれの手法に関する論文については、実はChatGPTの登場よりも前に発表されたものがあり時系列はバラバラなのですが、世間で注目されたタイミングをもとに解説していきます。

1.1.1 ■ LLMに指示を与える「プロンプト」

ChatGPTはその名のとおり、チャットへの入力を通じてLLMとやりとりします^{注1.3}。指示の仕方によりLLMはさまざまなタスクを実行します。

このような、**プロンプト**と呼ばれる指示をうまく与えることで、さまざまなタスクを実行できることがLLMの特徴です。一方でプロンプトは自然言語で記述するため自由度が高く、どのように書けばLLMが期待した動作をするかコントロールすることが課題になりました。

そうした中でキーワードとして登場したのが、**プロンプトエンジニアリング**です^{注1.4}。プロンプ

注 1.2 <https://chatgpt.com/>

注 1.3 「ChatGPTのおさらいと、プログラミングに活用するための第一歩」
<https://gihyo.jp/article/2023/03/programming-with-chatgpt-01>

注 1.4 「ChatGPTに指示する『プロンプトエンジニア』脚光」

トエンジニアリングは、LLMに期待した動作をさせるためのプロンプトを構築する技術を指します。

プロンプトが適切でない場合、LLMはあいまいな回答や一貫性のない出力、不適切な口調や形式といった問題のある挙動を示すことがあります。例えば、単に「マニュアルを作成して」と指示した場合、対象読者や内容の詳細度、形式が不明確なため、実際の業務では使いにくい汎用的な内容を生成してしまいます。これをプロンプトエンジニアリングにより、「新入社員向けの業務マニュアルを、箇条書き形式で、具体例を含めて1,000文字以内で作成してください」と具体化することで、明確で実用的なアウトプットを獲得できるようになります。

プロンプトエンジニアリングの手法は、どのLLMに対しても有効なものとは個別のLLM特有のものがあります。そのため、利用するLLMに対してどの手法が最適なのかどうかはドキュメントで確認することをおすすめします。Anthropic社のClaude CodeドキュメントではClaudeモデルに適用できる方法として「例を使用する」や「Claudeに役割を与える」などが紹介されています^{注1.5}。

プロンプトエンジニアリングの手法には、ある程度定まった「型」が存在します。毎回プロンプトを一から入力すると大変なので、この「型」をプロンプトテンプレートとして用意し、それを使ってプロンプトを構築する方法が広まってきました。Anthropic社は、Claude Codeの公式ドキュメントで、プロンプトエンジニアリングの手法の1つとしてプロンプトテンプレートを解説しています^{注1.6}。

なお、このプロンプトテンプレートという概念は、MCPではPromptsプリミティブ（後述）として標準化されています。

1.1.2 ■ LLMに外部情報を伝える「リソース」

プロンプトエンジニアリングの普及により、LLMを効率的に使えるようになってきました。

しかし、LLMは基本的には事前学習したデータをもとにして回答を生成しているため、たとえば特定の企業の社内情報を利用した回答を生成することはできません。

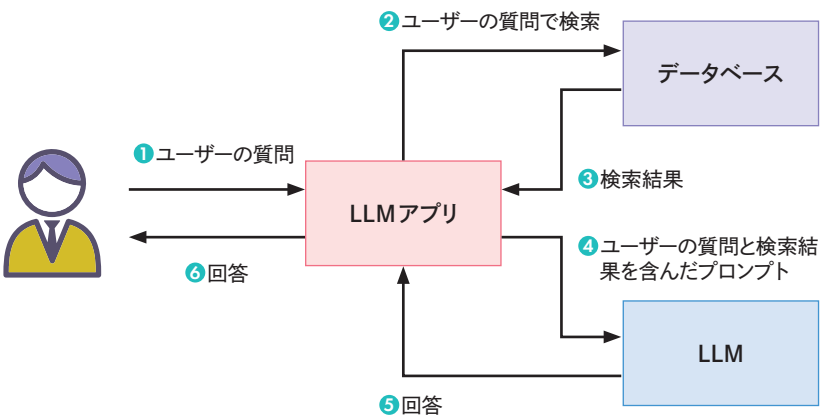
また、ナレッジカットオフと呼ばれるLLMがいつまでの情報をもとに学習したかを示す概念があります。LLMはこのナレッジカットオフ以降の出来事については正確な情報を持っていません。

こうした課題を解決する方法として、RAG (Retrieval Augmented Generation：検索拡張生成) が広まりました。

RAGにおいては、特定の企業の社内情報など通常のLLMには学習されていない知識を格納したデータベースを作成しておき、ユーザーの質問をもとにデータベースを検索します。LLMアプリ（LLMを活用したアプリケーション）がその検索結果をプロンプトに含めてLLMを呼び出し、LLMは検索結果を参照して回答を生成します。

言い換えると、RAGは、LLMアプリがユーザーの質問とデータベースから取得した検索結果をプロンプトテンプレートに当てはめてLLMを呼び出すことで、LLMが外部情報（＝リソース）を参照したうえで回答を生成する仕組みです。

■ RAGのイメージ



RAGのプロンプトテンプレートの例を以下に示します^{注1.7}。

■ RAGのプロンプトテンプレート例

あなたは質疑応答タスクのアシスタントです。以下の文脈情報を用いて質問に答えてください。答えがわからない場合は、「わからない」とだけ答えてください。回答は最大3文とし、簡潔にしてください。

Question: {question}
Context: {context}
Answer:

2023年後半から2024年頃、LLMの活用用途としてRAGは広く普及しました。AWS (Amazon Web Services) 社は2023年9月のAmazon Bedrockと併せて、RAGシステムの構築用埋め込みモデル「Titan Embeddings」の一般提供を開始しました^{注1.8}。なお、LLMのAmazon Titan Textモデルの一般提供開始は2023年の11月です。

^{注1.5} <https://www.nikkei.com/article/DGXZQOUC216YE0R20C23A4000000/>

^{注1.6} <https://docs.claude.com/ja/docs/build-with-claude/prompt-engineering/overview>

^{注1.7} <https://docs.claude.com/ja/docs/build-with-claude/prompt-engineering/prompt-templates-and-variables>

^{注1.8} <https://smith.langchain.com/hub/rllm/rag-prompt>にて公開されている内容を筆者が日本語に翻訳

^{注1.9} <https://aws.amazon.com/jp/blogs/news/amazon-bedrock-is-now-generally-available-build-and-scale-generative-ai-applications-with-foundation-models/>

今や、RAGはLLMの活用方法の1つとして、さまざまな企業内で利用が進んでいます。AWSではBedrockナレッジベースとしてRAGを簡単に構築する機能も提供されています。

このような、外部情報の提供機能は、MCPではResourcesプリミティブ（第2章を参照）として定義されています。

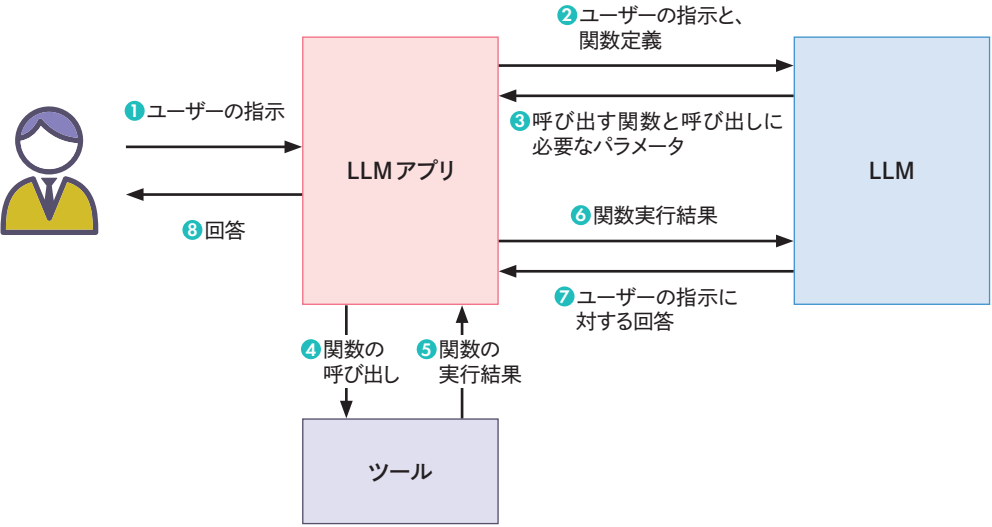
1.1.3 ■ LLMに外部とやりとりする手段を提供する「ツール」

RAGとともにLLMの活用方法として挙げられるのがAIエージェントです。AIエージェントとはユーザーの指示に基づいて自律的にタスクを実行するAIシステムのことです。

AIエージェントの実装にあたっては、重要な機能があります。2023年6月にOpenAI社が発表したFunction callingです^{注1.9}。

Function callingは、LLMの呼び出し時に関数定義をJSONで与えることで、呼び出す必要のある関数をLLMが判断し、さらに呼び出しに必要なパラメータを生成するというものです。これにより、LLMが動的に外部とやりとりができるようになりました。外部情報の「取得」だけでなく外部への「操作」も可能になった点が大きなポイントです。

Function callingの処理フロー



なお、Function callingには、さまざまな呼び方があります。本書ではAWSドキュメントの記述を優先し、Tool useと表記します。

注1.9 <https://platform.openai.com/docs/guides/function-calling/function-calling>

AWSで利用できるLLMとしては、2024年の3月に登場したClaude 3モデルでTool useに正式対応しました（Claude 2.1ではベータ扱いでした）。Claude 3.5 V2のリリースの際はマウスを操作したりテキストを入力するといったコンピュータを操作するツール「Computer use」、Claude 3.7リリースの際はファイル操作やターミナルコマンドの実行など複数のツールを備えたLLMアプリ「Claude Code」（第3章を参照）が同時に発表されるなど、LLM単体の性能だけではなくツール実行の性能も含めてアピールされています。

この外部ツール呼び出し機能は、MCPではToolsプリミティブ（第2章を参照）として標準化されています。

1.1.4 ■ 開発フレームワークの進化

LLMアプリはLLMを単体で使用するのではなくRAGで使用するベクトルデータベースやTool useで使用する外部ツールなどを組み合わせて実現します。各社さまざまなベクトルデータベースやツールを提供しており、それぞれに個別の実装を行うと開発規模が大きくなってしまいます。この課題を解決するためにLLMアプリ構築のフレームワークが必要とされ、最も人気のあるフレームワークの一つがLangChain^{注1.10}とLangGraphです。LangChainはLLMアプリ向け、LangGraphはAIエージェント向けといった位置づけです。LangChainでは1000以上のインテグレーションが提供されており、開発者は少ないコーディング量でLLMアプリを開発できるようになりました。

LangChainやLangGraphはあくまで開発者向けのものです。そのため、LangChainやLangGraphによりLLMアプリやAIエージェントを構築する際には「プロンプト」「リソース」「ツール」を用意しなければならず、プログラミング作業が必要でした。

そこで登場したのがMCPです。MCPは「プロンプト」「リソース」「ツール」を標準プロトコルとして統一し、異なるAIアプリ間で共有できるようにしたものです。なお、MCPで標準化された機能は他にもありますが、それらは第2章以降で触れます。

ここまでの出来事をまとめた表を以下に示します。

注1.10 <https://www.langchain.com/>

■ GPT-3.5リリースからMCPの発表までの主な出来事

日付	出来事
2022年11月30日	ChatGPT、GPT-3.5 リリース
2023年1月16日	LangChain 初回リリース (v0.0.64)
2023年3月14日	GPT-4 リリース
2023年3月23日	LangChain ブログ「Retrieval」投稿
2023年4月13日	Amazon Bedrock 発表
2023年6月13日	OpenAI 社から Function calling 登場
2023年9月28日	Amazon Bedrock 一般提供開始
2023年9月28日	Amazon Bedrock で Titan Embeddings が利用可能に
2023年11月21日	Claude 2.1 リリース (Tool Use にベータ対応)
2023年11月28日	Amazon Bedrock ナレッジベース、エージェント 一般提供開始
2024年3月4日	Claude 3 (Opus、Sonnet、Haiku) リリース (Tool Use 正式対応)
2024年6月20日	Claude 3.5 Sonnet リリース
2024年6月20日	LangGraph 初回リリース (0.1.23)
2024年10月22日	Claude 3.5 Sonnet V2、3.5 Haiku リリース (Computer Use 対応)
2024年11月7日	Amazon Bedrock プロンプト管理一般提供開始
2024年11月26日	MCP (Model Context Protocol) 発表、Claude Desktop がサポート開始

1.2 MCPが解決する課題とメリット

MCPはLLMアプリやAIエージェントを構築する際に共通して必要な「プロンプト」「リソース」「ツール」などの概念をプロトコルとして定義したものです。MCPで定義されているプリミティブ(機能)は、これまでなかった新しいものが登場したというよりも、ChatGPT 登場以降に登場したさまざまな概念を整理したものと言えます。

共通概念のプロトコル化によりさまざまなメリットがあります。

1.2.1 開発者の分離

プロトコル化により、LLMアプリやAIエージェントの開発者と、MCPサーバー(機能を提供する側)の開発者の分離が可能になりました。

たとえば、Web 検索機能が必要な LLM アプリを 2 つ開発するとします。これまではそれぞれ

の LLM アプリ内の機能として検索機能を開発していました。もちろん、ソースコードの流用が可能であれば移植できますが、開発言語が異なる場合はそうもいきません。

MCPがあれば、検索機能を一つのMCPサーバーとして開発し、2つのLLMアプリからMCPのプロトコルを通じて呼び出すことが可能になります。MCPはプロトコルのため開発言語が異なっても相互接続が可能です。

この開発者の分離は自社組織内に閉じた話ではありません。Web 検索サービスを事業として提供している Tavily 社や Brave 社は自社の検索サービスを呼び出せる MCP サーバーを提供しています。これまでは API という形態でインターフェイスを公開しておけば良かったのですが、LLM 時代においては MCP としてインターフェイスを公開することで、AI アプリから直接利用しやすくなるため重要になってきています。

LLM アプリや AI エージェントの開発者は個別に Web 検索機能を開発することなく、MCP で外部サービスと連携するだけで良くなります。一方で「Web 検索対応 AI アプリ」というだけでは LLM アプリや AI エージェントの差別化ができなくなり、業界や業務に特化した機能が求められるようになってきています。

1.2.2 一般ユーザーでも簡単に利用可能

MCPが注目を集めた重要なポイントとして、一般ユーザーでも簡単に利用できる点が挙げられます。

MCPの仕様が公開された際に、「開発に使用する SDK」以外に「Claude Desktop の MCP 対応」と「MCP サーバーのリファレンス実装」も同時に発表されました。Claude Desktop を設定するだけで、すでに公開されている MCP サーバーと連携できる状況で、MCP の有用性が誰でも理解できました。

この MCP の有用性にいち早く目をつけたベンダーは、コーディングエージェント(プログラムの作成や修正を支援する AI ツール) の MCP 対応を発表しました。開発者を「MCP を作る人」ではなく「MCP を使う人」として魅力を訴求することで MCP の有用性が広まっていきました。

LLM アプリ構築のフレームワークでも MCP 対応が進み、MCP の有用性を理解した開発者が、今度は MCP の開発者として MCP サーバーや MCP ホスト (MCP サーバーを呼び出す側) を開発していきました。このような好循環も作用し、MCP は瞬間に広まっていきました。

2026 年の時点では、有名なコーディングエージェントや AI 開発フレームワークは、ほぼすべてが MCP に対応している状況です。

以下、MCP 登場以降の出来事を年表形式で示します。

MCP登場以降の生成 AI アプリ開発に関する主な出来事

日付	出来事
2024年11月26日	MCP (Model Context Protocol) 発表、Claude Desktop がサポート開始
2024年12月13日	Cline が MCP をサポート (v2.2.0)
2025年1月23日	Cursor が MCP をサポート (0.45.11)
2025年2月24日	Claude 3.7 Sonnet リリース (Claude Code プレビュー開始)
2025年3月26日	MCP仕様の改定 (Version 2025-03-26)
2025年3月27日	OpenAI Agents SDK が MCP をサポート (v0.0.7)
2025年4月4日	GitHub Copilot がエージェントモードで MCP をプレビューサポート
2025年4月4日	Notion MCP サーバー公開 (1.0.0)
2025年4月5日	PayPal MCP サーバー公開 (0.1.0)
2025年4月5日	Google が Agent Development Kit が発表、MCP をサポート
2025年5月1日	Jira/Confluence MCP サーバー公開
2025年5月19日	Microsoft、Windows が MCP をネイティブサポートすると発表

1.3 AWSとMCP

開発者の間で MCP が注目される中、2025 年 4 月 1 日に **AWS MCP Servers** が発表されました^{注1.11}。

AWS MCP Servers は、AWS サービスとの連携を可能にする **MCP サーバーのコレクション**で、LLM アプリが AWS を最大限に活用できるよう支援します。AWS API 操作、インフラ管理、データベース操作など、幅広い AWS サービスへのアクセスを提供し、**すべてオープンソース**で公開されています^{注1.12}。

当初提供されていた MCP サーバーは以下の 5 つでした。

- Core
- CDK
- **Bedrock ナレッジベース**
- コスト解析
- Amazon Nova Canvas

注 1.11 <https://aws.amazon.com/jp/blogs/machine-learning/introducing-aws-mcp-servers-for-code-assistants-part-1/>
注 1.12 <https://awslabs.github.io/mcp/>

執筆時点では 60 以上の MCP サーバーが提供されています。

MCP ホストとしては、同月 2025 年 4 月に AWS 製のコーディングエージェントである **Amazon**

Q Developer CLI が MCP に対応しました (Amazon Q Developer の IDE 拡張も 6 月に対応)^{注1.13}。

また、**スペック** 駆動開発で注目を集めた **Kiro** も MCP に対応しています^{注1.14}。

そして 2025 年 5 月 17 日に AWS 製の AI エージェントフレームワーク「Strands Agents^{注1.15}」がオープンソースで公開されました^{注1.16}。Strands Agents は外部ツールを呼び出す方法として MCP を標準でサポートしています。

Strands Agents と同時に、Strands Agents のドキュメントを MCP サーバーとして提供する「Strands Agents MCP Server^{注1.17}」も公開されました。Amazon Q Developer CLI にこの Strands Agents MCP Server を接続することで、LLM が知らないはずの Strands Agents の使い方を MCP で与えることができます。これにより、Strands Agents を採用した AI エージェントの開発が正確に行えるようになります。

2025 年 7 月には AI エージェントや MCP サーバーを動作させる環境として「Amazon Bedrock AgentCore」が発表されました。2025 年 10 月には一般提供が開始されています^{注1.18}。Bedrock AgentCore を使うと、サーバーレスで AI エージェントや MCP サーバーをホスティングできます。また、AgentCore の Gateway 機能では、AWS Lambda 関数や **OpenAPI 仕様** を MCP サーバーに変換できます。

他に、AWS Marketplace (SaaS などのソリューションを販売できるサービス) では MCP サーバーの販売に対応するアップデート^{注1.19}がありました。

これらの AWS 製品・サービスは、それぞれ異なる用途で使い分けられます。

AWS の MCP 関連製品・サービス

AWS の MCP 関連製品・サービス	用途
AWS MCP Servers	AWS のさまざまなサービスと連携するための MCP サーバー
Kiro、Amazon Q Developer	MCP に対応したコーディングエージェント
Strands Agents	MCP に対応した AI エージェント開発フレームワーク
Amazon Bedrock AgentCore	AI エージェントや MCP サーバーをデプロイするサーバーレス環境
AWS Marketplace	MCP サーバーの購 ^{注1.19} 入に対応

注 1.13 <https://aws.amazon.com/jp/about-aws/whats-new/2025/04/amazon-q-developer-cli-model-context-protocol/>
なお、執筆時点では名称が **Kiro CLI** に変更されています。

注 1.14 <https://aws.amazon.com/jp/blogs/news/introducing-kiro/>

注 1.15 <https://strandsagents.com/latest/>

注 1.16 <https://aws.amazon.com/jp/blogs/news/introducing-strands-agents-an-open-source-ai-agents-sdk/>

注 1.17 <https://github.com/strands-agents/mcp-server>

注 1.18 <https://aws.amazon.com/jp/blogs/news/amazon-bedrock-agentcore-is-now-generally-available/>

注 1.19 <https://aws.amazon.com/jp/about-aws/whats-new/2025/07/ai-agents-tools-aws-marketplace/>

ここまで説明してきた、MCPに関するAWSの出来事を以下に年表で示します。

■MCPに関するAWSの主要な出来事

日付	出来事
2025年4月1日	AWS MCP Servers初回リリース
2025年4月29日	Amazon Q Developer CLIがMCPをサポート
2025年5月17日	Strands Agents 初回リリース (v0.1.0)、MCPをサポート
2025年6月13日	Amazon Q Developer IDE プラグインがMCPをサポート
2025年7月14日	Kiro プレビュー開始
2025年7月16日	Amazon Bedrock AgentCore プレビュー開始
2025年7月16日	AWS マーケットプレイスでAI エージェントとAI ツールの提供が開始
2025年7月16日	Amazon Bedrock AgentCore 一般提供開始

このように、AWSではMCPを利用するためのエコシステムがすでに確立されています。AWSでは、MCP利用者に対してはAWS MCP ServersがAWSのベストプラクティスに沿った開発を支援してくれます。MCP開発者に対してはフレームワークやデプロイ環境が整備されており、プロダクションリリース後の運用、さらに販売も視野に入れた環境が提供されています。

本書では、こうしたAWSのMCPに関するエコシステムを活用して、実践的なAIエージェント開発を解説します。